



UNIVERSIDAD DE CIENCIAS EMPRESARIALES Y SOCIALES

Carrera: Tecnicatura Universitaria en Programación

Asignatura: Programación II

Nombre del/a docente: Mario Daniel Detke

Nombre del/a estudiante: Gonzalo Ezequiel Beloqui

Fecha de entrega: 2026/04/24

Primer examen parcial: Propuesta de mejoras del sistema de turnos

I. Introducción

El desarrollo corresponde a un sistema de turnos médicos en Java, organizado en paquetes de modelo, controlador y principal, con la propuesta de incorporar una clase en el paquete vista para separar la interacción por consola, y desarrollado bajo el paradigma de la programación orientada a objetos, ya que utiliza clases para representar entidades, encapsulamiento de atributos, herencia en la clase Persona, una interfaz (Notificable) y persistencia en archivos de texto y XML mediante clases gestoras.

II. Análisis del desarrollo de versión base y propuesta de agregar, quitar o modificar clases

La versión base del sistema cuenta con las siguientes clases principales:

- I. "Persona" como clase abstracta base.
- II. "Paciente" y "Profesional" como especializaciones de "Persona"
- III. "Agenda" para representar la disponibilidad horaria de un profesional
- IV. "Turno" para representar una reserva concreta entre paciente y profesional
- V. "Notificacion" y "Notificable" para modelar el envío de avisos
- VI. "GestorPacientesTexto", "GestorProfesionalesTexto" y "GestorAgenda"
- VII. "Principal" como punto de entrada y clase de interacción por consola

La propuesta de mejora implementada amplía el sistema original mediante la incorporación de nuevas clases, validaciones y funcionalidades orientadas a completar el flujo real de gestión de turnos. En particular, se agregaron las clases Especialidad, GestorEspecialidadesTexto, GestorTurnosXML y VistaConsola, se reorganizó la lógica de Principal, se reforzaron las validaciones en Paciente, Profesional, Agenda, Turno, Persona y Notificacion, y se adecuó la persistencia de datos para acompañar estos cambios. A partir de estas modificaciones, el sistema ya no solo permite cargar pacientes, profesionales y agendas, sino también administrar especialidades, asignar turnos con restricciones concretas de edad, sexo, vigencia y disponibilidad horaria, impedir reservas inconsistentes o duplicadas, mostrar horarios libres y cancelar turnos futuros logrando una solución más completa.

II.1 Clases a agregar

I. GestorTurnosXML: Clase controladora encargada de la persistencia de turnos en formato XML, elegida en lugar de texto plano debido a que Turno es una entidad con una estructura más compleja y con asociaciones directas con otras clases del modelo, como Paciente, Profesional y Agenda. Este formato permite conservar una organización jerárquica más clara, facilita la identificación de cada dato almacenado y resulta más adecuado para representar relaciones entre objetos. En la implementación realizada, esta clase no solo guarda los turnos en XML, sino que también permite reconstruir cada turno a partir de los identificadores asociados a paciente, profesional y agenda, integrando de forma efectiva esta entidad al funcionamiento real del sistema.

II. VistaConsola: Se propone reducir la responsabilidad de la clase Principal, separando de ella la lógica de interacción por consola para ubicarla en una clase del paquete vista, de modo de lograr una mejor organización del sistema según el patrón vista-controlador trabajado en la materia. Esta organización permite distribuir responsabilidades de manera más clara entre las clases, favorece el mantenimiento del código y facilita futuras ampliaciones del sistema sin concentrar toda la lógica en la clase principal.

III. Especialidad: Con el objetivo de representar este concepto como una entidad propia del dominio y evitar que las especialidades queden cargadas como texto libre dentro de otras clases. A través de la clase GestorEspecialidadesTexto, se permite mantener un listado reutilizable y ampliable con persistencia en soporte .txt sin

necesidad de modificar el código fuente cada vez que se desee incorporar una nueva especialidad.

II.II Clases a modificar

I. Paciente: Se incorporaron los atributos "fechaNacimiento" y "sexo", junto con sus validaciones correspondientes. En particular, se controla que el DNI contenga solo números (7 u 8 dígitos), que el teléfono también sea numérico (entre 8 y 15 dígitos), que el email contenga al menos los caracteres "@" y ".", que la fecha de nacimiento respete el formato "dd/mm/aaaa" y represente una fecha válida de calendario, y que el sexo solo pueda tomar los valores "Femenino" o "Masculino" (restringiendo que los masculinos no puedan obtener turnos de especialidad ginecología. A partir de estos datos, el sistema determina que un paciente es pediátrico cuando, a la fecha del turno, su edad es menor a 18 años. Esta condición se utiliza luego para restringir la reserva de turnos en especialidades como pediatría.

II. Profesional: Se modificó para validar la matrícula, la especialidad y el email institucional. En particular, la matrícula debe contener una parte numérica exclusivamente y se normaliza agregando el prefijo MP. Por su parte, el email institucional queda restringido al dominio fijo "centrosalud.com". Además, el sistema controla que no existan dos profesionales con el mismo DNI, la misma matrícula o el mismo email institucional, garantizando unicidad en esos datos. La especialidad, como se mencionó en la descripción de dicha clase, a su vez, ya no se carga como texto libre, sino a partir de una lista persistida de especialidades disponibles.

III. Agenda: Se amplió su estructura incorporando fechaDesde y fechaHasta, de modo de definir la vigencia temporal de cada agenda. También se reforzaron las validaciones vinculadas al día y al horario de atención. En particular, el día de la semana ya no se ingresa libremente, sino que debe seleccionarse de una lista fija de Lunes a Sábado, excluyendo el domingo. En cuanto al horario, se estableció que la agenda solo puede configurarse dentro de la franja de atención entre 09:00 y 18:00, utilizando únicamente intervalos de 15 minutos. De este modo, la hora de inicio debe ser una fracción válida entre 09:00 y 17:45, la hora de fin entre 09:15 y 18:00, y además la hora inicial debe ser estrictamente menor que la hora final. También se controla que fechaDesde y fechaHasta tengan formato dd/mm/aaaa, representen fechas válidas y que la fecha inicial no sea posterior a la fecha final. Finalmente, se incorporó el control de superposición, evitando que un mismo profesional tenga dos agendas para el mismo día con horarios solapados.

IV. Turno: Dejó de ser una clase solo modelada para pasar a formar parte del flujo real del sistema. Se incorporaron validaciones de consistencia entre paciente, profesional, agenda, fecha, hora y estado. En particular, el turno solo puede asignarse si la agenda está en estado Activa, si la fecha elegida respeta el formato dd/mm/aaaa, corresponde a una fecha válida de calendario y no es anterior a la fecha actual, si además coincide con el día de la semana definido en la agenda y se encuentra dentro del rango de

vigencia establecido por fechaDesde y fechaHasta. La hora del turno no se ingresa manualmente como texto libre, sino que se selecciona a partir de la lista de horarios disponibles de la agenda, generada en intervalos de 15 minutos y excluyendo los ya ocupados. Además, se aplican restricciones de negocio según la especialidad: solo un paciente menor de 18 años al momento del turno puede acceder a pediatría; mientras que los turnos de ginecología solo pueden asignarse a pacientes de sexo femenino; y no se permite reservar más de un turno vigente o futuro de la misma especialidad para un mismo paciente. También se implementó la posibilidad de cancelar turnos futuros, cambiando su estado a anulado y liberando nuevamente ese horario para una nueva reserva.

V. Notificación: Se modificó para intervenir de forma concreta en el flujo del sistema mediante dos operaciones específicas: confirmación y anulación de turnos. En particular, se incorporaron métodos para preparar mensajes automáticos de confirmación y anulación, utilizando la fecha y la hora del turno como parte del contenido del mensaje. En ambos casos, el mensaje se envía tanto al paciente como al profesional asociado.

VI. Principal: Fue la clase con mayor nivel de modificación. Se reorganizó la lógica general del sistema, delegando la interacción por consola a VistaConsola e incorporando nuevas operaciones para especialidades, turnos y cancelaciones. También se centralizaron búsquedas, se encapsuló el cálculo de IDs a partir de los datos persistidos y se implementaron validaciones de flujo antes de confirmar operaciones. En la asignación de turnos, el sistema ahora solicita primero el paciente, luego filtra las especialidades permitidas según sus características, muestra solo las agendas válidas y finalmente presenta únicamente los horarios disponibles para la fecha elegida, permitiendo seleccionar el turno por número.

II.III Clases a eliminar

No se considera necesario eliminar clases del modelo actual, dado que las existentes responden correctamente al dominio planteado

III. Análisis de métodos en base a recursividad e iteración

El proyecto actual resuelve las búsquedas y recorridos de listas mediante estructuras iterativas, por ejemplo en los métodos de búsqueda de pacientes, profesionales y agendas, así como en la obtención del siguiente identificador de agenda. Si bien técnicamente todos los métodos podrían reescribirse de forma recursiva, no se considera que esa decisión aporte una mejora real en este caso. La recursividad resulta más adecuada cuando el problema presenta una estructura naturalmente recursiva, como árboles, jerarquías o subdivisión del problema en partes equivalentes. En este sistema, los datos se almacenan en listas lineales obtenidas desde archivos, por lo que el recorrido secuencial es más simple, más legible y más coherente con el nivel de complejidad del proyecto. Asimismo, la iteración permite mantener un control

más explícito del recorrido y de las condiciones de corte, lo que resulta conveniente en métodos que dependen de validaciones puntuales y de la lectura de datos persistidos en archivos.